

模拟目录管理功能

题目描述

实现一个模拟 **目录管理** 功能的软件，输入一个命令序列，输出最后一条命令运行结果。

支持命令：

1. 创建目录命令：mkdir 目录名称，如 mkdir abc 为在当前目录创建abc目录，如果已存在同名目录则不执行任何操作。此命令无输出。
2. 进入目录命令：cd 目录名称，如 cd abc 为进入abc目录，特别地，cd .. 为返回上级目录，如果目录不存在则不执行任何操作。此命令无输出。
3. 查看当前所在路径命令：pwd，输出当前路径字符串。

约束：

1. 目录名称仅支持小写字母；mkdir 和 cd 命令的参数仅支持单个目录，如：mkdir abc 和 cd abc；不支持嵌套路径和绝对路径，如 mkdir abc/efg，cd abc/efg，mkdir /abc/efg，cd /abc/efg 是不支持的。
2. 目录符号为/，根目录/作为初始目录。
3. 任何不符合上述定义的无效命令不做任何处理并且无输出。

输入描述

输入 N 行 **字符串** ，每一行字符串是一条命令。

输出描述

输出最后一条命令运行结果字符串。

备注

命令行 数限制100行以内，目录名称限制10个字符以内。

用例

输入	mkdir abc cd abc pwd
输出	/abc/
说明	在根目录创建一个abc的目录并进入abc目录中查看当前目录路径，输出当前路径/abc/。

题目解析

本题感觉主要是考察树形结构定义，以及逻辑模拟。

目录结构，其实就是一个树形结构。一个父目录下面可以有多个直接子目录，而一个子目录只能有一个父目录。因此，本题需要定义出一个多叉树结构。

关于树节点定义如下：

```
class TreeNode {
    String dicName; // 当前目录的名字
    TreeNode father; // 当前目录的父目录
    List<TreeNode> children; // 当前目录的子目录
}
```

接下来，就是实现目录管理能力：

- mkdir
- cd
- pwd

实现这三个能力前，我们需要定义出树结构：

```
class Tree {  
    TreeNode root; // 树的根目录  
    TreeNode cur; // 当前所在目录  
}
```

其中，tree.cur 用于指向当前所在目录，初始时 tree.cur = tree.root。

mkdir，其实就是在 tree.cur 目录下创建一个子目录，但是前提是 tree.cur 下面不存在对应子目录名，否则不操作。mkdir操作不改变 tree.cur 指向。

cd，有两种情况：

1. cd ..

cd .. 是返回上级目录（父目录），但是前提是 tree.cur.father 存在，否则不操作。如果 tree.cur.father 存在，则cd .. 会改变 tree.cur = tree.cur.father。

2. cd 目录名

cd 目录名 是进入子目录，但是前提是 tree.cur 包含对应目录名的子目录，否则不操作。如果 存在对应子目录，则 cd操作会改变 tree.cur = 对应子目录

pwd 是输出当前目录的路径字符串，我们可以不停进行 tree.cur = tree.cur.father 的倒序遍历，获取遍历过程中目录名tree.cur.dicName，直到 tree.cur == NULL。最后拼接时注意反转。

上面是目录管理功能的三个能力大致实现思路，具体实现根据不同语言有不同改进，比如 cd 目录名 功能，我们需要遍历 tree.cur.children 来确认对应目录名是否存在，这里Java，JS，Py定义 tree.cur.children 时可以使用 字典Map 来定义（key是目录名，val是对应目录名的TreeNode），从而实现快速查找对应目录。

另外，本题需要对mkdir, cd, pwd的命令参数做约束：

- mkdir, cd 的参数只能是1个, 如果超过1个, 就不进行操作
- pwd 不需要参数, 如果有参数, 就不进行操作
- mkdir, cd的参数 (目录名) 只能由小写字母组成, 否则不操作
- mkdir, cd的参数 (目录名) 不能是嵌套路径, 或者绝对路径

JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6    class TreeNode {
7      constructor(curDicName, father) {
8        this.curDicName = curDicName;
9        this.father = father;
10       this.children = {};
11     }
12   }
13
14   class Tree {
15     constructor() {
16       // root是根目录, 根目录 / 作为初始目录
17       this.root = new TreeNode("/", null);
18       // cur用于指向当前正在操作的目录
19       this.cur = this.root;
20     }
21
22     mkdir(dicName) {
23       // mkdir 目录名称, 如 mkdir abc 为在当前目录创建abc目录, 如果已存在同名目录则不执行任何操作
24       if (!this.cur.children[dicName]) {
25         this.cur.children[dicName] = new TreeNode(dicName + "/", this.cur);
26       }
27     }
28   }
29 })();
```

运行

```
28
29 |     cd(dicName) {
30 |         if (dicName == "..") {
31 |             // cd .. 为返回上级目录，如果目录不存在则不执行任何操作
32 |             if (this.cur.father != null) {
33 |                 // cur 变更指向上级目录
34 |                 this.cur = this.cur.father;
35 |             }
36 |         } else {
37 |             // cd 目录名称，如 cd abc 为进入abc目录，如果目录不存在则不执行任何操作
38 |             if (this.cur.children[dicName]) {
39 |                 // cur 变更指向下级目录
40 |                 this.cur = this.cur.children[dicName];
41 |             }
42 |         }
43 |     }
44
45 |     pwd() {
46 |         // 输出当前路径字符串
47 |         const arr = [];
48
49 |         // 倒序路径，即不停向上找父目录
50 |         let cur = this.cur;
51 |         while (cur != null) {
52 |             arr.push(cur.curDicName);
53 |             cur = cur.father;
54 |         }
55
56 |         // 反转后拼接
57 |         return arr.reverse().join("");
58 |     }
59 | }
60
61 | // 初始化目录结构
62 | const tree = new Tree();
63 | // 记录最后一条命令的输出
64 | let lastCommandOutPut = "/";
```

```
65 |
66 |   outer: while (true) {
67 |     try {
68 |       const line = await readline();
69 |
70 |       // 本地测试解开此行
71 |       // if (line == "") break;
72 |
73 |       const tmp = line.split(" ");
74 |       const cmd_key = tmp[0];
75 |
76 |       if (cmd_key == "pwd") {
77 |         // pwd 命令不需要参数
78 |         if (tmp.length != 1) continue;
79 |         lastCommandOutPut = tree.pwd();
80 |       } else if (cmd_key == "mkdir" || cmd_key == "cd") {
81 |         // 约束: mkdir 和 cd 命令的参数仅支持单个目录, 如: mkdir abc 和 cd abc
82 |         if (tmp.length != 2) continue;
83 |
84 |         // 目录名
85 |         const cmd_val = tmp[1];
86 |
87 |         if (!(cmd_key == "cd" && cmd_val == "..")) {
88 |           // 目录名约束校验
89 |           // 约束: 目录名称仅支持小写字母
90 |           // 约束: 不支持嵌套路径和绝对路径, 如 mkdir abc/efg, cd abc/efg, mkdir /abc/efg, cd /abc/efg 是不支持的。
91 |           // 关于嵌套路径和绝对路径, 我简单理解就是cmd_val含有'/'字符, 可以被小写字母判断涵盖住
92 |           for (let c of cmd_val) {
93 |             if (c < "a" || c > "z") continue outer;
94 |           }
95 |         }
96 |
97 |         if (cmd_key == "mkdir") {
98 |           tree.mkdir(cmd_val);
99 |           // 题目要求输出最后一个命令的运行结果, 因此, 对于无输出的命令, 我认为需要覆盖掉前面的命令的输出结果
100 |           lastCommandOutPut = "/";
101 |         } else {
```

```

102 |         tree.cd(cmd_val);103 |
    | // 题目进要求输出最后一个命令的运行结果，因此，对于无输出的命令，我认为需要覆盖掉前面的命令的输出结果104 |
    | lastCommandOutPut = "/";105 |         }
106 |     }
107 | } catch (e) {
108 |     break;
109 | }
110 | }
111 |
112 | console.log(lastCommandOutPut);
113 | })();

```

Java算法源码

```

1 | import java.util.HashMap;
2 | import java.util.Scanner;
3 |
4 | public class Main {
5 |     static class TreeNode {
6 |         String curDicName;
7 |         TreeNode father;
8 |         HashMap<String, TreeNode> children;
9 |
10 |         public TreeNode(String curDicName, TreeNode father) {
11 |             this.curDicName = curDicName;
12 |             this.father = father;
13 |             this.children = new HashMap<>();
14 |         }
15 |     }
16 |
17 |     static class Tree {
18 |         TreeNode root;
19 |         TreeNode cur;
20 |

```

```
21 public Tree() {
22     // root是根目录，根目录 / 作为初始目录
23     this.root = new TreeNode("/", null);
24     // cur用于指向当前正在操作的目录
25     this.cur = root;
26 }
27
28 public void mkdir(String childDicName) {
29     // mkdir 目录名称，如 mkdir abc 为在当前目录创建abc目录，如果已存在同名目录则不执行任何操作
30     this.cur.children.putIfAbsent(
31         childDicName, new TreeNode(childDicName + "/", this.cur)); // 目录符号为 /
32 }
33
34 public void cd(String dicName) {
35     if (dicName.equals("..")) {
36         // cd .. 为返回上级目录，如果目录不存在则不执行任何操作
37         if (this.cur.father != null) {
38             // cur 变更指向上级目录
39             this.cur = this.cur.father;
40         }
41     } else {
42         // cd 目录名称，如 cd abc 为进入abc目录，如果目录不存在则不执行任何操作
43         if (this.cur.children.containsKey(dicName)) {
44             // cur 变更指向下级目录
45             this.cur = this.cur.children.get(dicName);
46         }
47     }
48 }
49
50 public String pwd() {
51     // 输出当前路径字符串
52     StringBuilder sb = new StringBuilder();
53
54     // 倒序路径，即不停向上找父目录
55     TreeNode cur = this.cur;
56     while (cur != null) {
57         // 头插目录名，保证路径中目录层级正确
```



```

58         sb.insert(0, cur.curDicName); 59         cur = cur.father;
60     }
61
62     return sb.toString();
63 }
64 }
65
66 public static void main(String[] args) {
67     Scanner sc = new Scanner(System.in);
68
69     // 初始化目录结构
70     Tree tree = new Tree();
71
72     // 记录最后一条命令的输出
73     String lastCommandOutPut = "/";
74
75     outer:
76     while (sc.hasNextLine()) {
77         String line = sc.nextLine();
78
79         // 本地测试解开此行
80         //      if (line.equals("")) break;
81
82         String[] tmp = line.split(" ");
83
84         String cmd_key = tmp[0];
85
86         if (cmd_key.equals("pwd")) {
87             // pwd 命令不需要参数
88             if (tmp.length != 1) continue;
89             lastCommandOutPut = tree.pwd();
90         } else if (cmd_key.equals("mkdir") || cmd_key.equals("cd")) {
91             // 约束: mkdir 和 cd 命令的参数仅支持单个目录, 如: mkdir abc 和 cd abc
92             if (tmp.length != 2) continue;
93
94             // 目录名

```

```

95         String cmd_val = tmp[1]; 96
97         if (!(cmd_key.equals("cd") && cmd_val.equals(".."))) {
98             // 目录名约束校验
99             // 约束: 目录名称仅支持小写字母
100            // 约束: 不支持嵌套路径和绝对路径, 如 mkdir abc/efg, cd abc/efg, mkdir /abc/efg, cd /abc/efg 是不支持的。
101            // 关于嵌套路径和绝对路径, 我简单理解就是cmd_val含有 '/' 字符, 可以被小写字母判断涵盖住
102            for (int i = 0; i < cmd_val.length(); i++) {
103                char c = cmd_val.charAt(i);
104                if (c < 'a' || c > 'z') continue outer;
105            }
106        }
107
108        if (cmd_key.equals("mkdir")) {
109            tree.mkdir(cmd_val);
110
111            // 题目进要求输出最后一个命令的运行结果, 因此, 对于无输出的命令, 我认为需要覆盖掉前面的命令的输出结果
112            lastCommandOutPut = "/";
113        } else {
114            tree.cd(cmd_val);
115            // 题目进要求输出最后一个命令的运行结果, 因此, 对于无输出的命令, 我认为需要覆盖掉前面的命令的输出结果
116            lastCommandOutPut = "/";
117        }
118    }
119 }
120
121 System.out.println(lastCommandOutPut);
122 }
123 }

```

Python算法源码

```

1 class TreeNode:
2     def __init__(self, curDicName, father):
3         self.curDicName = curDicName
4         self.father = father

```

```

5         self.children = {}
6
7
8 class Tree:
9     def __init__(self):
10         # root是根目录，根目录 / 作为初始目录
11         self.root = TreeNode("/", None)
12         # cur用于指向当前正在操作的目录
13         self.cur = self.root
14
15     def mkdir(self, dicName):
16         # mkdir 目录名称，如 mkdir abc 为在当前目录创建abc目录，如果已存在同名目录则不执行任何操作
17         self.cur.children.setdefault(dicName, TreeNode(dicName + "/", self.cur)) # 目录符号为 /
18
19     def cd(self, dicName):
20         if dicName == "..":
21             # cd .. 为返回上级目录，如果目录不存在则不执行任何操作
22             if self.cur.father is not None:
23                 # cur 变更指向上级目录
24                 self.cur = self.cur.father
25         else:
26             # cd 目录名称，如 cd abc 为进入abc目录，如果目录不存在则不执行任何操作
27             if self.cur.children.get(dicName) is not None:
28                 # cur 变更指向下级目录
29                 self.cur = self.cur.children[dicName]
30
31     def pwd(self):
32         # 输出当前路径字符串
33         lst = []
34
35         # 倒序路径，即不停向上找父目录
36         cur = self.cur
37         while cur is not None:
38             lst.append(cur.curDicName)
39             cur = cur.father
40
41         # 反转后拼接

```

```
42 |         lst.reverse()
43 |         |
44 |         return "".join(lst)
45 |
46 |
47 | # 算法逻辑
48 | # 初始化目录结构
49 | tree = Tree()
50 |
51 | # 记录最后一条命令的输出
52 | lastCommandOutput = "/"
53 |
54 | while True:
55 |     try:
56 |         line = input()
57 |
58 |         # 本地测试解开此行
59 |         # if line == "":
60 |         #     break
61 |
62 |         tmp = line.split()
63 |         cmd_key = tmp[0]
64 |
65 |         if cmd_key == "pwd":
66 |             # pwd 命令不需要参数
67 |             if len(tmp) != 1:
68 |                 continue
69 |             lastCommandOutput = tree.pwd()
70 |         elif cmd_key == "mkdir" or cmd_key == "cd":
71 |             # 约束: mkdir 和 cd 命令的参数仅支持单个目录, 如: mkdir abc 和 cd abc
72 |             if len(tmp) != 2:
73 |                 continue
74 |
75 |             # 目录名
76 |             cmd_val = tmp[1]
77 |
78 |             # 目录名约束校验
```

```

79 |         # 约束：目录名称仅支持小写字母80 |
      |
      | # 约束：不支持嵌套路径和绝对路径，如 mkdir abc/efg, cd abc/efg, mkdir /abc/efg, cd /abc/efg 是不支持的。81 |
      | # 关于嵌套路径和绝对路径，我简单理解就是cmd_val含有 '/' 字符，可以被小写字母判断涵盖住82 |
      | if not (cmd_val.isalpha() and cmd_val.islower()) and not (cmd_key == 'cd' and cmd_val == '..'):83 |
      |     continue84 |
85 |     if cmd_key == "mkdir":
86 |         tree.mkdir(cmd_val)
87 |         # 题目进要求输出最后一个命令的运行结果，因此，对于无输出的命令，我认为需要覆盖掉前面的命令的输出结果
88 |         lastCommandOutput = "/"
89 |     else:
90 |         tree.cd(cmd_val)
91 |         # 题目进要求输出最后一个命令的运行结果，因此，对于无输出的命令，我认为需要覆盖掉前面的命令的输出结果
92 |         lastCommandOutput = "/"
93 | except:
94 |     break
95 |
96 | print(lastCommandOutput)

```

C算法源码

```

1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 |
5 | /** 树节点 **/
6 | typedef struct TreeNode {
7 |     char curDicName[11]; // 当前目录名
8 |     struct TreeNode *father; // 父目录（只能有一个）
9 |     struct LinkedList *children; // 子目录（可以有多个，这里使用链表记录）
10 | } TreeNode;
11 |
12 | /** 链表节点 **/
13 | typedef struct ListNode {
14 |     TreeNode *ele; // 链表用于记录多个子目录，因此链表节点的内容就是树节点

```

```

15     struct ListNode *next; 16 } ListNode;
17
18 /** 链表 **/
19 typedef struct LinkedList {
20     ListNode *head;
21     ListNode *tail;
22     int size;
23 } LinkedList;
24
25 /** 树 **/
26 typedef struct Tree {
27     TreeNode *root; // 记录树根节点
28     TreeNode *cur; // 记录当前目录对应的节点
29 } Tree;
30
31 /** 链表结构方法 **/
32 // 初始化链表
33 LinkedList *new_LinkedList() {
34     LinkedList *link = (LinkedList *) malloc(sizeof(LinkedList));
35
36     link->head = NULL;
37     link->tail = NULL;
38     link->size = 0;
39
40     return link;
41 }
42
43 // 尾插链表
44 void addLast_LinkedList(LinkedList *link, TreeNode *ele) {
45     ListNode *listNode = (ListNode *) malloc(sizeof(ListNode));
46     listNode->ele = ele;
47     listNode->next = NULL;
48
49     if (link->size == 0) {
50         link->head = listNode;
51         link->tail = listNode;
52     } else {

```

```

53         link->tail->next = listNode;
54         link->tail = listNode;
55     }
56
57     link->size++;
58 }
59
60 // 遍历链表，获取指定节点的内容
61 TreeNode *get_LinkedList(LinkedList *link, char *dicName) {
62     ListNode *curListNode = link->head;
63     while (curListNode != NULL) {
64         if (strcmp(curListNode->ele->curDicName, dicName) == 0) {
65             return curListNode->ele;
66         }
67         curListNode = curListNode->next;
68     }
69     return NULL;
70 }
71
72 /** 树形结构方法 */
73 TreeNode *new_TreeNode(char *curDicName, TreeNode *father) {
74     TreeNode *treeNode = (TreeNode *) calloc(1, sizeof(TreeNode));
75
76     strcpy(treeNode->curDicName, curDicName);
77     treeNode->father = father;
78     treeNode->children = new_LinkedList();
79
80     return treeNode;
81 }
82
83 // 初始化树
84 Tree *new_Tree() {
85     Tree *tree = (Tree *) malloc(sizeof(Tree));
86
87     // 由于目录名结尾都要带 '/', 因此可以认为根目录是空串，后期拼接时再尾部追加 '/'
88     // 另外根目录没有父目录，因此父目录设置为NULL
89     tree->root = new_TreeNode("", NULL);

```

```
90 // 初始时, 当前目录就是根目录 91 | tree->cur = tree->root;
92
93 return tree;
94 }
95
96 // 创建指定目录
97 void mkdir_Tree(Tree *tree, char *dicName) {
98     TreeNode *p = get_LinkedList(tree->cur->children, dicName);
99
100     // mkdir 目录名称, 如 mkdir abc 为在当前目录创建abc目录, 如果已存在同名目录则不执行任何操作
101     if (p != NULL) {
102         return;
103     }
104
105     TreeNode *treeNode = new_TreeNode(dicName, tree->cur);
106     addLast_LinkedList(tree->cur->children, treeNode);
107 }
108
109 // 跳转到指定目录
110 void cd_Tree(Tree *tree, char *dicName) {
111     if (strcmp(dicName, "..") == 0) {
112         // cd .. 为返回上级目录, 如果目录不存在则不执行任何操作
113         if (tree->cur->father != NULL) {
114             // cur 变更指向上级目录
115             tree->cur = tree->cur->father;
116         }
117     } else {
118         TreeNode *p = get_LinkedList(tree->cur->children, dicName);
119
120         // cd 目录名称, 如 cd abc 为进入abc目录, 如果目录不存在则不执行任何操作
121         if (p != NULL) {
122             // cur 变更指向下级目录
123             tree->cur = p;
124         }
125     }
126 }
127
```



```
128 | // 输出当前路径字符串
    |
    | 129 | char *pwd_Tree(Tree *tree) {
130 |     char *tmp = (char *) calloc(10000, sizeof(char));
131 |     char *res = (char *) calloc(10000, sizeof(char));
132 |
133 |     // 倒序路径，即不停向上找父目录
134 |     TreeNode *cur = tree->cur;
135 |     while (cur != NULL) {
136 |         strcpy(tmp, res);
137 |
138 |         strcpy(res, cur->curDicName);
139 |         strcat(res, "/");
140 |         strcat(res, tmp);
141 |         cur = cur->father;
142 |     }
143 |
144 |     return res;
145 | }
146 |
147 | int main() {
148 |     // 初始化目录结构
149 |     Tree *tree = new_Tree();
150 |
151 |     // 记录最后一条命令的输出
152 |     char lastCommandOutPut[10000] = {'/'};
153 |
154 |     char s[50];
155 |     while (gets(s)) {
156 |         // 本地测试解开此行注释
157 |         if (strlen(s) == 0) break;
158 |
159 |         char *cmd_key = strtok(s, " ");
160 |         char *cmd_val = strtok(NULL, " ");
161 |
162 |         if (strcmp(cmd_key, "pwd") == 0) {
163 |             // pwd 命令不需要参数
164 |             if (cmd_val != NULL) {
```

```

165         continue;
166     }
167     strcpy(lastCommandOutPut, pwd_Tree(tree));
168 } else if (strcmp(cmd_key, "mkdir") == 0 || strcmp(cmd_key, "cd") == 0) {
169     // 约束: mkdir 和 cd 命令的参数仅支持单个目录, 如: mkdir abc 和 cd abc
170     if (cmd_val == NULL) continue;
171
172     char *p = strtok(NULL, " ");
173     if (p != NULL) continue;
174
175     if (!(strcmp(cmd_key, "cd") == 0 && strcmp(cmd_val, "..") == 0)) {
176         unsigned long long len = strlen(cmd_val);
177
178         // 目录名约束校验
179         // 约束: 目录名称仅支持小写字母
180         // 约束: 不支持嵌套路径和绝对路径, 如 mkdir abc/efg, cd abc/efg, mkdir /abc/efg, cd /abc/efg 是不支持的。
181         // 关于嵌套路径和绝对路径, 我简单理解就是cmd_val含有 '/' 字符, 可以被小写字母判断涵盖住
182         int i = 0;
183         for (; i < len; i++) {
184             char c = cmd_val[i];
185
186             if (c < 'a' || c > 'z') {
187                 break;
188             }
189         }
190
191         if (i != len) {
192             continue;
193         }
194     }
195
196     if (strcmp(cmd_key, "mkdir") == 0) {
197         mkdir_Tree(tree, cmd_val);
198         // 题目进要求输出最后一个命令的运行结果, 因此, 对于无输出的命令, 我认为需要覆盖掉前面的命令的输出结果
199         memset(lastCommandOutPut, '\0', strlen(lastCommandOutPut));
200         lastCommandOutPut[0] = '/';
201     } else {

```

```

202         cd_Tree(tree, cmd_val);203
    // 题目进要求输出最后一个命令的运行结果，因此，对于无输出的命令，我认为需要覆盖掉前面的命令的输出结果204
    memset(lastCommandOutPut, '\0', strlen(lastCommandOutPut));205         lastCommandOutPut[0] = '/';
206     }
207 }
208 }
209
210 puts(lastCommandOutPut);
211
212 return 0;
213 }

```

C++ 算法源码

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5  public:
6      string curDicName{};
7      TreeNode *father{};
8      map<string, TreeNode *> children{};
9
10     TreeNode(string curDicName, TreeNode *father) : curDicName(std::move(curDicName)), father(father) {}
11 };
12
13 class Tree {
14 public:
15     TreeNode *root{}; // root是根目录，根目录 / 作为初始目录
16     TreeNode *cur{}; // cur用于指向当前正在操作的目录
17
18     Tree() {
19         root = new TreeNode("/", nullptr);
20         cur = root;

```

```

21 } 22 |
23 void mkdir(string &childDicName) const {
24     // mkdir 目录名称, 如 mkdir abc 为在当前目录创建abc目录, 如果已存在同名目录则不执行任何操作
25     if (cur->children.count(childDicName) == 0) {
26         cur->children[childDicName] = new TreeNode(childDicName + "/", cur); // 目录符号为 /
27     }
28 }
29
30 void cd(string &dicName) {
31     if (dicName == "..") {
32         // cd .. 为返回上级目录, 如果目录不存在则不执行任何操作
33         if (cur->father != nullptr) {
34             // cur 变更指向上级目录
35             cur = cur->father;
36         }
37     } else {
38         // cd 目录名称, 如 cd abc 为进入abc目录, 如果目录不存在则不执行任何操作
39         if (cur->children.count(dicName) > 0) {
40             // cur 变更指向下级目录
41             cur = cur->children[dicName];
42         }
43     }
44 }
45
46 string pwd() const {
47     // 输出当前路径字符串
48     string sb;
49
50     // 倒序路径, 即不停向上找父目录
51     TreeNode *curNode = this->cur;
52     while (curNode != nullptr) {
53         // 头插目录名, 保证路径中目录层级正确
54         sb = curNode->curDicName + sb;
55         curNode = curNode->father;
56     }
57
58     return sb;

```

```

59     }
60     };
61
62 int main() {
63     // 初始化目录结构
64     Tree tree;
65
66     // 记录最后一条命令的输出
67     string lastCommandOutPut = "/";
68
69     do {
70         string s;
71         getline(cin, s);
72
73         stringstream ss(s);
74         string token;
75
76         vector<string> tmp;
77
78         while (getline(ss, token, ' ')) {
79             tmp.emplace_back(token);
80         }
81
82         string cmd_key = tmp[0];
83
84         bool flag = false;
85
86         if (cmd_key == "pwd") {
87             // pwd 命令不需要参数
88             if (tmp.size() != 1) continue;
89             lastCommandOutPut = tree.pwd();
90         } else if (cmd_key == "mkdir" || cmd_key == "cd") {
91             // 约束: mkdir 和 cd 命令的参数仅支持单个目录, 如: mkdir abc 和 cd abc
92             if (tmp.size() != 2) continue;
93
94             // 目录名
95             string cmd_val = tmp[1];

```

```

96 |
97 |         if (!(cmd_key == "cd" && cmd_val == "..")) {
98 |             // 目录名约束校验
99 |             // 约束：目录名称仅支持小写字母
100 |             // 约束：不支持嵌套路径和绝对路径，如 mkdir abc/efg, cd abc/efg, mkdir /abc/efg, cd /abc/efg 是不支持的。
101 |             // 关于嵌套路径和绝对路径，我简单理解就是cmd_val含有 '/' 字符，可以被小写字母判断涵盖住
102 |             for (const auto &c: cmd_val) {
103 |                 if (c < 'a' || c > 'z') {
104 |                     flag = true;
105 |                 }
106 |             }
107 |
108 |             if (flag) continue;
109 |         }
110 |
111 |         if (cmd_key == "mkdir") {
112 |             tree.mkdir(cmd_val);
113 |             // 题目进要求输出最后一个命令的运行结果，因此，对于无输出的命令，我认为需要覆盖掉前面的命令的输出结果
114 |             lastCommandOutPut = "/";
115 |         } else {
116 |             tree.cd(cmd_val);
117 |             // 题目进要求输出最后一个命令的运行结果，因此，对于无输出的命令，我认为需要覆盖掉前面的命令的输出结果
118 |             lastCommandOutPut = "/";
119 |         }
120 |     }
121 | } while (!cin.eof());
122 |
123 | cout << lastCommandOutPut << endl;
124 |
125 | return 0;
126 | }

```